Programovanie v C++: Smerníky, referencie, funkcie

Nízkoúrovňová správa pamäte a adresácia, modulárny návrh, abstrakcia, rekurzia

Programovanie mechatronických aplikácií
ÚAMT FEI STU

PMEAP Prednáška 2

Premenné z pohľadu fyzickej pamäte

Pri deklarácii premennej vykoná systém na pozadí dôležitý proces:

- Počítač asociuje názov premennej s konkrétnou **fyzickou adresou (memory location)** v operačnej pamäti.
- Následne do tohto špecifického bloku pamäte uloží priradenú hodnotu.

Pri referovaní na premennú pomocou jej názvu, počítač vykonáva v skutočnosti dva kroky:

1. Vyhľadá (look up) adresu v pamäti, ktorej zodpovedá názov premennej.
2. Pristúpi na danú adresu a načíta alebo zapíše hodnotu, ktorú pamäť obsahuje.

Operátory prístupu k pamäti (& a *)

Jazyk C++ umožňuje programátorom inžiniersky rozložiť tento proces prístupu k pamäti a ovládať jeho kroky nezávisle prostredníctvom operátorov.

Získanie adresy (Address-of operator)

Operátor `&` aplikovaný na premennú `x` (zápis `&x`) nevyhodnocuje jej hodnotu, ale vráti **adresu premennej x v pamäti**.

Dereferencovanie (Dereference operator)

Operátor `*` aplikovaný na adresu (zápis `*(&x)`) vezme danú adresu, pristúpi do pamäte a načíta z nej fyzickú hodnotu (dereferencuje ju). Zápis `*(&x)` je logicky ekvivalentný volaniu samotnej premennej `x`.

Motivácia: Prečo využívať smerníky (Pointers)?

Pamäťové adresy, nazývané v kontexte jazyka C/C++ **smerníky (pointers)**, umožňujú manipulovať s dátami vysoko flexibilným spôsobom. Manipulácia s adresami dát je často oveľa výpočtovo **efektívnejšia**, než presúvanie a kopírovanie samotných dát.
Výhody smerníkov:

- Flexibilnejšia realizácia paradigmy **pass-by-reference**.
- Efektívna správa komplexných dátových štruktúr, aj ak sú ich dáta fyzicky rozptýlené v rozdielnych pamäťových blokoch (fragmentácia).
- Využitie **polymorfizmu** - volanie funkcií nad dátami bez exaktnej vedomosti o ich presnom type na úrovni kompilácie (bude súčasťou neskorších prednášok).

Architektúra smerníka

Čo je to presne smerník?

- Z hardvérového hľadiska sú smerníky iba bežné premenné, ktoré ukladajú celé čísla (integers).
- Zásadný rozdiel je v sémantike: Tieto "celé čísla" náhodne zodpovedajú špecifickým **adresám pamäte** (zvyčajne adresám iných premenných v systéme).
- Ak smerník obsahuje adresu premennej x, hovoríme, že smerník **ukazuje na** x (points to x).

Prístup: Hodnotu samotnej premennej x dokážeme načítať alebo zmeniť tak, že smerník takzvane "dereferencujeme".

Vizuálna pamäťová mapa

Pamäť procesora (RAM) si abstraktne vizualizujeme ako rad susediacich buniek (blocks of memory), pričom každá bunka reprezentuje jeden blok.

Adresa	12309	12310	12311	...	**12314**
Identifikátor	`ptr`				`x`
Hodnota	**12314**				*(dáta)*

V príklade vyššie premenná `ptr` obsahuje uložené číslo 12314. Keďže ide o adresu, na ktorej fyzicky leží premenná `x`, notáciou určujeme, že `ptr` **ukazuje** na `x`.

Syntax deklarácie smerníka

Na deklaráciu premennej smerníka, ktorá bude ukazovať napríklad na celočíselnú premennú, používame operátor * pri deklarácii.

```
1 int x = 42;
2 int *ptr = &x;
```

Deklaracia a inicializacia smernika

- `int *ptr` oznámi kompilátoru, že ide o "smerník na integer" (pointer to an integer).
- `= &x` tento smerník **inicializuje** priamo fyzickou adresou premennej `x`.

Smerníky môžeme vytvoriť pre akýkoľvek známy dátový typ. Všeobecná šablóna je: `data_type *pointer_name;`.

Praktické použitie dereferencovania (l-hodnoty)

Akonáhle je smerník nastavený na platnú pamäť, môžeme operátor * aplikovať na smerník kedykoľvek pre prístup k uloženým dátam.

```
1 // Vypise hodnotu x (v nasom pripade 42)
2 cout << *ptr;
3
4 // Dereferencovany smernik funguje ako "l-value"
5 // Tento zapis fyzicky prepise hodnotu v premennej x!
6 *ptr = 5;
```

Dôležitý fakt: Bez operátora * sa identifikátor `ptr` odvoláva len na **samotný smerník** (hexadecimálnu adresu), a nie na hodnotu, na ktorú smeruje. Zápis `cout << ptr;` by vypísal adresu v pamäti v 16-tkovej sústave.

Smerníky vo funkciách (Simulácia Referencie)

Smerníky sa dajú odovzdávať do funkcií ako argumenty podobne ako základné typy. To umožňuje upravovať originálne hodnoty volajúcej premennej (podobne ako pri odovzdávaní referenciou).

```
1  void squareByPtr(int *numPtr) {
2      // Vsimnite si nasobne pouzitie dereferencie
3      *numPtr = *numPtr * *numPtr;
4  }
5
6  int main() {
7      int x = 5;
8      squareByPtr(&x); // Do funkcie posielame ADRESU premennej x
9      cout << x;       // Vypise: 25
10 }
```

Modifikátory const a smerníky

Kľúčové slovo `const` (konštanta) môžeme vo vzťahu k smerníkom umiestniť na dve rôzne miesta. Dôvodom je, že **obmedzujeme zmenu dvoch rôznych premenných**: obmedzujeme buď samotný smerník (adresu) alebo hodnotu, na ktorú ukazuje.

Smerník na konštantu (Zmeniteľná adresa, nemenné dáta)

```
1 const int *ptr;
```

Môžete smerníku priradiť iný pamäťový blok, ale nesmiete pomocou dereferencovania meniť hodnotu celého čísla v pamäti.

Modifikátory const a smerníky (pokračovanie)

Konštantný smerník (Nemenná adresa, zmeniteľné dáta)

```
1 int * const ptr = &x;
```

Deklaruje konštantný smerník. Môžete meniť celočíselné dáta pomocou smerníka (`*ptr = 10;`), ale samotnému smerníku už nikdy nesmiete priradiť inú adresu (nesmie ukazovať inam).

Striktná konštanta (Nemenné všetko)

```
1 const int * const ptr = &x;
```

Zakazuje meniť tak adresu v smerníku, ako aj dátovú hodnotu v pamäti.

Nebezpečenstvá: Nulové a deallokované smerníky

Null Pointers: Ak je akýkoľvek smerník nastavený na hodnotu `0`, nazýva sa Nulový smerník (null pointer). Keďže žiadna adresa `0` v bežnom operačnom systéme nie je k dispozícii pre používateľa, je to zámerne **neplatný smerník**. Často slúži ako signál, že smerník zatiaľ na nič neukazuje. Pokus o jeho dereferenciu (napr. `*ptr = 5;`) spôsobí okamžitý pád programu (runtime error - segmentation fault).
Deallocated Pointers (Dangling):

```
1 int *myFunc() {
2     int phantom = 4;
3     return &phantom; // CHYBA! phantom zanikne po skonceni funkcie.
4 }
```

Pri návrate funkcie sa premenná `phantom` dealokuje a adresa bude neplatná.

Zavedenie referencií (References)

Referencie slúžia v C++ ako alternatíva k smerníkom. Vytvárajú čistý **alias (iné meno)** pre už existujúcu pamäť.

```
1 int y = 10;
2 int &x = y; // Premenna 'x' je alias pre 'y'
3 x = 20;     // Toto priamo zmeni hodnotu premennej 'y'
```

- Pri lokálnej deklarácii musí byť referencia (na rozdiel od smerníka) vždy povinne **inicializovaná** k inej premennej priamo pri zrode.
- Ak sa zmení alias (`x`), okamžite to ovplyvní originál (`y`), pretože sú to len dve mená pre ten istý fyzický kus hardvéru.

Rozdiel: Smerník verzus Referencia

Na úrovni kompilátora a procesora (assembly) sa referencie implementujú úplne rovnako ako smerníky. Sú to de-facto smerníky, ktoré sú **automaticky dereferencované pri každom použití**.
Zásadné pravidlá (rozdielnosti):

1. **Syntax:** Referencie sú v kóde implicitne pre-dereferencované. Nemusíte manuálne písať operátor $*$ pred premennú.
2. **Imutabilita cieľa:** Zatiaľ čo smerník môžete počas behu presmerovať na úplne iný kus pamäte, referenciu nie. Alias **nemožno zmeniť na iný blok**, akonáhle bol raz vytvorený.
3. **Prijímanie argumentu:** Keď inicializujeme referenciu, neposkytujeme operátor adresy & k zdrojovej premennej (ako pri smerníkoch).

Syntaktické preťaženie symbolov: * a &

Zápisy pre začiatočníkov v C++ sú mätúce, lebo operátory * a & majú dvojitý význam na základe kontextu.

Operátor * má dva významy:

- Pri *deklarácii typu* označuje, že premenná je typologicky **smerník**.
- Vo *výraze* aplikovaný na smerník predstavuje **dereferenciu** pre načítanie/zápis hodnôt.

Operátor & má taktiež dva významy:

- Pri *deklarácii typu* označuje, že typ premennej je **referencia** (napr. `int &x;`).
- Vo *výraze* reprezentuje operáciu, ktorá **vyextrahuje pamäťovú adresu** (napr. `&x`).

Synergia smerníkov a polí

Základný postulát

Názov poľa (array name) je z pohľadu kompilátora C++ de-facto iba konštantný **smerník ukazujúci na jeho prvý prvok**.

Dôsledky v praxi:

- Zápis `myArray[3]` technicky hovorí kompilátoru: "Presuň sa o 3 pamäťové miesta ďalej od začiatočnej počiatočnej adresy `myArray`".
- Toto presne vysvetľuje dôvod, prečo **indexy polí začínajú nulou**. Nultý prvok znamená, že na nájdenie prvku sa od začiatku (od adresy samotného poľa) musíme posunúť o nulu (0) krokov.
- Vzhľadom k tomu sú polia do funkcií automaticky odovzdávané **referenciou**, pretože neodosielame celé pole, ale len smerník na jeho začiatok.

Smerníková aritmetika (Pointer Arithmetic)

Pri manipulácii s poliami C++ umožňuje vykonávať matematické operácie sčítania a odčítania priamo so samotnými adresami pamäte (pointer arithmetic). Pridanie čísla `n` k smerníku posunie ukazovateľ o `n` **pozícií** hlbšie do pamäte.

```
1 long arr[] = {6, 0, 9, 6};
2 long *ptr = arr;   // ptr ukazuje na nulty prvok (hodnotu 6)
3 ptr++;             // posun smernika
4
5 long *ptr2 = arr + 3; // ptr2 ukazuje na posledny prvok (6)
```

Bezpečnosť typov: Step Size

Kompilátor C++ disponuje veľkou automatizáciou. Pri zápise `ptr++` by neškolený programátor mohol predpokladať, že sa posunie len o 1 byte (čo by narušilo formát dát).

- V skutočnosti systém automaticky zistí typ a pripočíta patričný **veľkostný krok (step size)**. Ak je prvok (napríklad `long`) dlhý 8 bytov, kompilátor pridá k adrese procesora číslo 8, čím adresuje skutočne "ďalší prvok" poľa.
- Podobne to funguje s odčítaním adresy. `ptr2 - ptr` nevráti matematický rozdiel pamäťových blokov, ale korektný **počet array elementov (prvkov poľa)** vložených medzi nimi, v našom prípade hodnotu 2.

Pointer-Subscript vs Pointer-Offset notácia

Z tohto dôvodu sú smerníky a polia úplne zameniteľné. Na adresáciu v poliach existujú syntaktické alternatívy.

1. Zápis s hranatými zátvorkami (Subscript notation): Zápis na ktorý sme bežne zvyknutí, použiteľný pre polia a zameniteľne i pre samotné smerníky.

```
1 myArray[3] = 42;
```

2. Zápis Pointer-Offset: Explicitne posunieme adresu o 3 a následne celý výraz manuálne dereferencujeme pomocou zátvoriek a operátora *.

```
1 *(myArray + 3) = 42;
```

Tieto dva zápisy sa prekladajú na 100% identické procesorové inštrukcie.

Dátové typy reťazcov: char* (String)

Reťazec `char*` je z pamäťového hľadiska len smerník ukazujúci na **prvý prvok poľa znakov**. Je tu avšak striktný bezpečnostný rozdiel:

```
1 // Vytvara lokalne pole (Array). Obsah JE mozne neskor modifikovat.
2 char courseName1[] = {'6', '.', '0', '9', '6', '\0'};
3
4 // String literal. Nacita sa do "Read-Only" pamate pri starte programu.
5 char *courseName2 = "6.096";
```

Pokus o úpravu textu v `courseName1` je absolútne korektný. Avšak pokus o prepísanie znakov priamo v ukazovateli `courseName2` vedie obvykle ku chybe pri kompilácii (alebo zlyhá počas behu a program padne), nakoľko sa pokúšame zapísať do uzamknutej **Read-Only Memory** segmentov OS.

FUNKCIE Motivácia: Anti-pattern “Copy-Paste“ programovanie

Častou chybou začiatočníkov je implementácia rovnakého algoritmu viackrát v zdrojovom kóde.

```
// Vypocet 3^4
int threeExpFour = 1;
for (int i = 0; i < 4; i = i + 1) {
    threeExpFour = threeExpFour * 3;
}
cout << "3^4 is " << threeExpFour << endl;

// Vypocet 6^5 (Duplikacia logiky!)
int sixExpFive = 1;
for (int i = 0; i < 5; i = i + 1) {
    sixExpFive = sixExpFive * 6;
}
cout << "6^5 is " << sixExpFive << endl;
```

Takýto prístup (“Copy-paste coding“) je technicky chybný a vedie k neudržateľným systémom.

Inžiniersky pohľad: Prečo definovať vlastné funkcie?

- **Čitateľnosť (Readability):** Zápis `sqrt(5)` je oveľa jasnejší ako copy-pastovanie celého algoritmu na výpočet druhej odmocniny priamo do výkonného kódu.
- **Udržateľnosť (Maintainability):** Ak potrebujete zmeniť algoritmus (napríklad optimalizovať zložitosť), stačí zmeniť implementáciu funkcie na jednom mieste namiesto stoviek miest, kde bol kód skopírovaný.
- **Znovupoužiteľnosť kódu (Code reuse):** Umožňuje zapuzdriť logiku tak, aby implementované algoritmy mohli bezpečne využívať aj iní programátori v iných moduloch.

Riešenie: Abstrakcia pomocou funkcií

Namiesto duplikovania cyklov definujeme všeobecnú funkciu, ktorá vypočíta ľubovoľnú mocninu.

```
int threeExpFour = raiseToPower(3, 4);
cout << "3^4 is " << threeExpFour << endl;

int sixExpFive = raiseToPower(6, 5);
cout << "6^5 is " << sixExpFive << endl;

int twelveExpTen = raiseToPower(12, 10);
cout << "12^10 is " << twelveExpTen << endl;
```

Kód je kratší, sémanticky jasný a plne abstrahuje samotný proces iterácie od volajúceho (caller).

Anatómia funkcie: Syntaktická štruktúra

Každá funkcia v C++ sa skladá z presne definovaných častí kompilovanej signatúry a tela.

```
1 int raiseToPower(int base, int exponent) {
2     int result = 1;
3     for (int i = 0; i < exponent; i = i + 1) {
4         result = result * base;
5     }
6     return result;
7 }
```

- `int`: Návratový typ (Return type).
- `raiseToPower`: Názov funkcie (Function name).
- `(int base, int exponent)`: Argumenty s určeným typom.

Signatúra funkcie a jej poradie

Základné pravidlo argumentov

Záleží na poradí argumentov (Argument order matters). Kompilátor priraďuje hodnoty k premenným zľava doprava na základe ich pozície.

Pre funkciu `raiseToPower(int base, int exponent)`:

- `raiseToPower(2, 3)` reprezentuje výpočet $2^3 = 8$.
- `raiseToPower(3, 2)` reprezentuje výpočet $3^2 = 9$.

Spojenie názvu funkcie a zoznamu typov jej argumentov tvorí tzv. **signatúru funkcie** (Function signature).

Telo funkcie a inštrukcia return

- **Telo (Body):** Zložené zátvorky {} obklopujú logiku funkcie.
- **Return statement:** Inštrukcia `return` ukončuje beh funkcie a posiela výsledok späť volajúcemu.

```
1 int raiseToPower(int base, int exponent) {
2     int result = 1;
3     for (int i = 0; i < exponent; i = i + 1) {
4         result = result * base;
5     }
6     return result; // Navratova instrukcia
7 }
```

Návratové typy (Return types)

- Funkcia môže vrátiť **maximálne jednu hodnotu**.
- Typ vrátenej hodnoty musí byť identický alebo implicitne konvertovateľný na deklarovaný návratový typ funkcie.

```
1 int foo() {
2     return "hello"; // ERROR: Typ mismatch (char* vs int)
3 }
4
5 char* bar() {
6     return "hello"; // OK: Korektny typ
7 }
```

Absencia návratovej hodnoty: Typ void

- Ak funkcia nemá vracať žiadnu hodnotu (vykonáva iba nejakú akciu, napr. výpis do konzoly), priraďuje sa jej typ `void`.

```
1 void printNumber(int num) {
2     cout << "number is " << num << endl;
3 }
4
5 int main() {
6     printNumber(4); // Vypise: number is 4
7     return 0;
8 }
```

Poznámka: Nemôžete deklarovať inštanciu/premennú typu `void` (napr. `void x;` zlyhá pri kompilácii).

Riadenie toku pomocou early return

Inštrukcia `return` sa nemusí nachádzať výlučne na konci funkcie. Beh funkcie sa okamžite ukončí hneď pri prvom vykonanom príkaze `return`.

```
1 void printNumberIfEven(int num) {
2     if (num % 2 == 1) {
3         cout << "odd number" << endl;
4         return; // Okamzite ukoncenie funkcie
5     }
6     cout << "even number; number is " << num << endl;
7 }
```

Typová kontrola argumentov (Type-checking)

Jazyk C++ je silne typový. Typy argumentov poskytnutých pri volaní sa musia zhodovať so signatúrou.

```
1 void printOnNewLine(int x) {
2     cout << x << endl;
3 }
```

- `printOnNewLine(3)` funguje bez problémov.
- `printOnNewLine("hello")` sa neskompiluje (kompilačná chyba).

Zmena argumentov ovplyvňuje použiteľnosť

Ak prepíšeme funkciu pre iný typ, jej správanie a pravidlá kompilácie sa menia.

```
1 void printOnNewLine(char *x) {
2     cout << x << endl;
3 }
```

- V tomto prípade `printOnNewLine(3)` zlyhá kompilátor (nie je konverzia z `int` na pointer textu).
- `printOnNewLine("hello")` sa úspešne skompiluje a funguje.

Polymorfizmus kompilátora: Preťažovanie funkcií (Overloading)

C++ umožňuje mať viacero funkcií s **rovnakým názvom**, ale **rozdielnymi argumentmi**.

```
1 void printOnNewLine(int x) {
2     cout << "Integer: " << x << endl;
3 }
4
5 void printOnNewLine(char *x) {
6     cout << "String: " << x << endl;
7 }
```

Kompilátor automaticky vyberie správnu funkciu na základe dátových typov argumentov, ktoré dodáme pri volaní (Invocation).

Ako kompilátor rieši Overloading?

Pri volaní s rôznymi parametrami:

- `printOnNewLine(3)` zavolá verziu pre `int` a vypíše ‘‘`Integer: 3`‘‘.
- `printOnNewLine("hello")` zavolá verziu pre reťazec a vypíše ‘‘`String: hello`‘‘.

Technický detail: Kompilátor implementuje tzv. **Name Mangling**. Do skutočného názvu funkcie v binárnom súbore zakóduje typy jej argumentov (napr. `_Z14printOnNewLinei` vs `_Z14printOnNewLinePc`), vďaka čomu linker vie presne, čo má pripojiť.

Preťažovanie počtom argumentov

Funkcie môžeme preťažovať nielen zmenou typov, ale aj zmenou počtu argumentov.

```
1 void printOnNewLine(int x) {
2     cout << "1 Integer: " << x << endl;
3 }
4
5 void printOnNewLine(int x, int y) {
6     cout << "2 Integers: " << x << " and " << y << endl;
7 }
```

- Volanie `printOnNewLine(3)` vypíše ‘‘`1 Integer: 3`‘‘.
- Volanie `printOnNewLine(2, 3)` vypíše ‘‘`2 Integers: 2 and 3`‘‘.

Kompilácia jedným priechodom (Single-pass limitation)

Prekladač C++ spracováva kód zhora nadol. Deklarácie funkcií musia existovať predtým, ako k nim pristúpime (pred volaním).

```
1 int foo() {
2     return bar() * 2; // ERROR: bar() este nebol deklarovany
3 }
4
5 int bar() {
6     return 3;
7 }
```

Tento kód vyvolá kompilačnú chybu, pretože kompilátor nevie nič o funkcii `bar()` počas kompilácie funkcie `foo()`.

Riešenie 1: Zmena poradia (Topological Sort)

Najjednoduchším riešením je presunúť implementáciu volanej funkcie pred tú, ktorá ju volá.

```
1 int bar() {
2     return 3;
3 }
4
5 int foo() {
6     return bar() * 2; // OK: bar() je uz znama
7 }
```

Avšak, v rozsiahlych systémoch alebo pri kruhových závislostiach (circular dependencies) toto riešenie nie je možné.

Riešenie 2: Prototypy funkcií (Forward Declarations)

Prototyp funkcie informuje kompilátor o existencii signatúry (názov a typy) skôr, ako dodáme samotnú implementáciu.

```
1 int bar(); // Funkcny prototyp
2
3 int foo() {
4     return bar() * 2; // OK: Kompilator slubuje, ze implementaciu najde neskor
5 }
6
7 int bar() {
8     return 3; // Skutocna implementacia (telo)
9 }
```

Flexibilita prototypov

- Prototypy funkcií musia zhodovať návratový typ a typy argumentov.
- Názvy argumentov v prototype sú voliteľné a nezáleží na nich (môžu sa vynechať úplne).

```
1  int square(int); // Platny prototyp
2  int square(int x); // Tiez platny prototyp
3  int square(int z); // Tiez plati, nezalezi na mene
4
5  int cube(int x) {
6      return x * square(x);
7  }
8
9  int square(int x) { // Finalna implementacia
10     return x * x;
11 }
```

Oddelenie deklarácie a implementácie (Modulárny dizajn)

V profesionálnom programovaní sa prototypy ukladajú do separátnych hlavičkových súborov (header files). To oddeľuje špecifikáciu funkcie (API) od jej implementácie.

myLib.h (Hlavičkový súbor - Interface)

```
1 // Obsahuje prototypy
2 int square(int);
3 int cube(int);
```

myLib.cpp (Zdrojový súbor - Implementácia)

```
1 #include "myLib.h"
2
3 int cube(int x) { return x * square(x); }
4 int square(int x) { return x * x; }
```

Knižnice a distribučný model C++

- Knižnice sú zvyčajne distribuované vo forme hlavičkového súboru (`.h` s prototypmi) a **binárneho súboru** (`.dll` na Windows, `.so` na Linux/Mac).
- Tento model umožňuje distribúciu (kompilovanej) implementácie bez nutnosti zdieľať privátny `.cpp` zdrojový kód.
- Koncový užívateľ knižnice potrebuje iba poznať funkčné prototypy v hlavičkovom súbore, nie implementačný kód v `.cpp`.

Úloha Linker-a pri prepájaní

Proces zostavenia C++ projektu pozostáva z Preprocesora, Kompilátora a Linkera.

Linker

Linker (súčasť sady kompilátorov) sa stará o nájdenie skutočnej implementácie funkcií v pripojených `.dll` alebo `.obj` súboroch počas času zostavovania kódu (compile/link time).

Ak v kóde zavoláme funkciu `squareRoot()` deklarovanú v hlavičke, no chýba jej binárna knižnica, nevznikne kompilačná chyba, ale fatálna **Linker Error** (napr. *Unresolved external symbol*).

Rekurzia: Matematický základ

Funkcie môžu v C++ volať samé seba. Tento koncept sa nazýva rekurzia. Z inžinierskeho hľadiska sa s každým volaním uloží na call stack nový "Stack Frame".
Rekurzia je výborná na dekompozíciu matematických definícií, napr. Fibonacciho postupnosť definovaná vzťahom: $fib(n) = fib(n-1) + fib(n-2)$.

Štruktúra rekurzie: Base Case a Recursive Step

Každá rekurzívna funkcia musí obsahovať dve kľúčové časti:

1. **Base case (Základný stav):** Podmienka, ktorá ukončí rekurzívne zanáranie, zvyčajne statický návrat hodnoty (aby nedošlo k nekonečnému cyklu).
2. **Recursive step (Rekurzívny krok):** Výpočet, ktorý zahŕňa volanie tej istej funkcie s modifikovanými parametrami bližšie k základnému stavu.

Príklad: Implementácia Fibonacciho postupnosti

Priame vyjadrenie rekurzívnej funkcie z jej matematického vzorca:

```cpp
int fibonacci(int n) {
    if (n == 0 || n == 1) { // Base case
        return 1;
    } else {
        // Recursive step
        return fibonacci(n - 2) + fibonacci(n - 1);
    }
}
```

Varovanie pre inžinierov: Pre veľké n je takýto naivný stromový prístup neefektívny ($\mathcal{O}(2^n)$ časová zložitosť) a hrozí **Stack Overflow**, ak nie je aplikovaná optimalizácia (napr. dynamické programovanie).

Scope (Rozsah platnosti)

V správe pamäti určuje **Scope** (Rozsah), odkiaľ v programe je možné k premennej pristupovať. Záleží na tom, na akom mieste (v akom bloku) bola premenná deklarovaná.
Typy rozsahov platnosti v C++:

- Globálny scope (Global scope)
- Lokálny/Funkčný scope (Function scope)
- Blokový scope (Block scope - vo vnútri slučiek a podmienok)

Globálne premenné (Global Variables)

Ak potrebujeme zachovať stav naprieč viacerými volaniami viacerých funkcií, môžeme vytvoriť globálnu premennú. Tieto premenné sa alokujú v statickej/BSS dátovej sekcii pamäte a je k nim možné pristupovať z ktorejkoľvek funkcie.

```
1  int numCalls = 0; // Globalna premenna
2
3  void foo() {
4      ++numCalls; // Zvysi sa zakazdym, ked je foo() zavolana
5  }
6
7  int main() {
8      foo(); foo(); foo();
9      cout << numCalls << endl; // Vypise 3
10 }
```

Funkčný scope (Function Scope)

Každá funkcia si spravuje svoje vlastné, oddelené premenné (alokované na procesorovom zásobníku - Stack).

```
int raiseToPower(int base, int exponent) {
    int result = 1; // "result" existuje iba tu
    // ...
    return result;
}

int max(int num1, int num2) {
    int result; // Toto je UPLNE INA premenna "result"
    // ...
    return result;
}
```

Vďaka izolovanému funkčnému scope môže mať každá funkcia vlastnú, separátnu premennú s názvom `result`.

Rozsah platnosti cyklov a podmienok (Block Scope)

- Slučky (`for`, `while`) a podmienky (`if`, `else`) tiež vytvárajú vlastný scope pre premenné definované v ich vnútri.
- Premenné deklarované vo vnútri (ako počítadlo `i` v `for` cykle) **zanikajú hneď po opustení bloku**.

```
1 for (int i = 0; i < 30; i = i + 1) {
2     double estimate = (high + low) / 2;
3     // ...
4 }
5 // Tu (po skonceni loopu) "estimate" ani "i" NEEXISTUJU
```

Chyby pri nedodržaní rozsahu platnosti

Nemôžete pristupovať k premenným, ktoré sa nachádzajú "mimo rozsah" (out of scope).

```
for (int i = 0; i < 30; i = i + 1) {
    double estimate = (high + low) / 2;
}
// ERROR: Pristup k "estimate" zlyha pri kompilacii,
// premenna je mimo scope!
return estimate;
```

Riešenie: Premennú musíme deklarovať (alokovať pamäť) v nadradenom scope (vo vyššom bloku pred cyklom).

Predávanie parametrov: Pass by value vs Pass by reference

Ako presne systém C++ presúva dáta medzi volajúcou (main) a volanou (foo) funkciou? Máme dva hlavné prístupy:

1. **Pass by Value (Predávanie hodnotou):** Vytvorí sa striktná a izolovaná **kópia** obsahu.
2. **Pass by Reference (Predávanie odkazom):** Vytvorí sa **ukazovateľ/alias**, obe funkcie modifikujú fyzicky ten istý pamäťový register/adresu.

Predávanie parametrov hodnotou (Pass by Value)

Doteraz sme všetky argumenty odovzdávali hodnotou. Systém pre túto funkciu **vyrobí kópiu** pôvodnej premennej. Zmeny v lokálnej kópii neovplyvnia pôvodnú hodnotu.

```
1  void increment(int a) {
2      a = a + 1; // Modifikuje IBO LOKALNU kopiu!
3      cout << "a in increment " << a << endl;
4  }
5
6  int main() {
7      int q = 3;
8      increment(q); // Posielame hodnotu (3), neposielame pamat "q"
9      cout << "q in main " << q << endl; // q je STALE 3!
10 }
```

Predávanie parametrov odkazom (Pass by Reference)

Ak chcete modifikovať **originálnu premennú** a predísť drahému kopírovaniu pamäte, predávame argument odkazom. V signatúre pridáme symbol & k dátovému typu.

```
// vsimnite si ampersand &
void increment(int &a) {
    a = a + 1;
    cout << "a in increment " << a << endl;
}

int main() {
    int q = 3;
    increment(q); // Modifikuje samotnu premennu "q" v pamati main()
    cout << "q in main " << q << endl; // Vypise: q in main 4 (FUNGUJE!)
}
```

Prípadová štúdia z praxe: Funkcia Swap

Bez odovzdania odkazom by bolo nemožné napísať funkciu `swap()`, ktorá fyzicky prehodí hodnoty dvoch premenných volajúceho v pamäti.

```
1  void swap(int &a, int &b) {
2      int t = a;
3      a = b;
4      b = t;
5  }
6
7  int main() {
8      int q = 3;
9      int r = 5;
10     swap(q, r);
11     cout << "q " << q << endl; // Vypise q 5
12     cout << "r " << r << endl; // Vypise r 3
13 }
```

Analýza stavu pamäte pri funkcii Swap

Krok za krokom podla:

1. `main` definuje priestor pre `q`=3 a `r`=5.
2. Funkcia `swap` sa zavolá a jej lokálne "alias" odkazy `a` a `b` ukazujú priamo na RAM bloky `q` a `r`.
3. Lokálna premenná `t` vo funkcii `swap` sa naplní hodnotou 3.
4. `a` (čo je fyzicky `q`) dostáva hodnotu z `b` (z `r`), `a`=5, teda v `main` je `q`=5.
5. `b` (čo je fyzicky `r`) dostane uloženú hodnotu z `t`, mení sa na `r`=3.

Viacnásobné návratové hodnoty pomocou referencií

Štandardne, inštrukcia `return` umožňuje vrátiť len **jednu hodnotu**. Predávaním výstupných (output) premenných cez odkaz (&) môžeme túto limitáciu prekonať a vrátiť viacero výstupov súčasne.

```
1  int divide(int num, int den, int &remainder) {
2      remainder = num % den; // Toto zostane zapisane v premennej volajuceho
3      return num / den;      // Hlavna navratova hodnota (podiel)
4  }
5
6  int main() {
7      int num = 14, den = 4, rem;
8      int result = divide(num, den, rem); // Zisti vysledok A ZAROVEN zvysok
9      cout << result << "*" << den << "+" << rem << "=" << num << endl;
10     // Vypise: 3*4+2=14
11 }
```

Smerníky vo funkciách (Simulácia Referencie)

Smerníky sa dajú odovzdávať do funkcií ako argumenty podobne ako základné typy. To umožňuje upravovať originálne hodnoty volajúcej premennej (podobne ako pri odovzdávaní referenciou).

```
1  void squareByPtr(int *numPtr) {
2      // Vsimnite si nasobne pouzitie dereferencie
3      *numPtr = *numPtr * *numPtr;
4  }
5
6  int main() {
7      int x = 5;
8      squareByPtr(&x); // Do funkcie posielame ADRESU premennej x
9      cout << x;       // Vypise: 25
10 }
```

Polia ako argumenty funkcií

Polia môžeme posielať do funkcií ako argumenty. Pri deklarácii funkcie uvedieme parameter ako pole **bez zadania dimenzie**.

```
// Vsimnite si, ze specifikujeme aj dlzku (length) pola
int sum(const int array[], const int length) {
    long sum = 0;
    for(int i = 0; i < length; sum += array[i++]);
    return sum;
}

// Volanie v main():
int arr[] = {1, 2, 3, 4, 5, 6, 7};
cout << "Sum: " << sum(arr, 7) << endl;
```

Sémantika odovzdávania do funkcií (Pass-by-Reference)

Kritický technický detail

V jazyku C/C++ sa polia do funkcií **odovzdávajú odkazom (passed by reference)**. Kód vo funkcii manipuluje priamo s pôvodným pamäťovým priestorom, nie s jeho kópiou.

- Technicky sa odovzdáva smerník (pointer) na prvý prvok poľa.
- **Dôsledok:** Akékoľvek zmeny, ktoré vykonáte na poli vo vnútri funkcie, **sa prejavia aj v pôvodnom volajúcom prostredí (calling scope)**.
- Aj preto pole z predchádzajúceho príkladu použilo kvalifikátor `const int array[]`, čím funkcii explicitne zakazujeme pole modifikovať.

Záverečné informácie a Standard Library (cmath)

- V praxi nepotrebujeme manuálne programovať zložité matematické algoritmy ako napríklad na výpočet mocniny (raiseToPower) alebo druhej odmocniny (squareRoot).
- Jazyk C++ má rozsiahlu štandardnú knižnicu (Standard Library). Pre pokročilú matematiku stačí vložiť (include) knižnicu <cmath>.
- Táto hlavička nám prináša optimalizované, na úrovni hardvéru kompilované funkcie ako pow() pre mocninu a sqrt() pre druhú odmocninu.

```
1 #include <cmath>
2
3 double fourthRoot(double num) {
4     return sqrt(sqrt(num));
5 }
```

Zhrnutie: Funkcie - Nástroj abstrakcie

- Funkcie zvyšujú čitateľnosť kódu, udržateľnosť a možnosť opakovaného použitia logiky (DRY - Don't Repeat Yourself).
- Kompilačné sekvencie si vyžadujú dopredné špecifikácie rozhraní formou prototypov pred samotným volaním.
- Modulárny prístup pomocou `.h` (interfejs) a `.cpp/.dll` (implementácia) rozdeľuje softvér do stabilných blokov.
- Rekurzia uľahčuje matematický zápis na úkor správy pamäte.
- Pamäťový Scope zaisťuje ochranu lokálnych dát (enkapsuláciu logiky).
- Správa referencií (`&`) odkrýva priamy prístup do blokov RAM, šetrí výpočtový čas kopírovania a umožňuje multihodnotové návraty.

Q & A

Ďakujem za pozornosť!